



# Alexander Eckerlin

AI Enablement, interne KI-Tools, Automatisierung | Senior Managing Editor

Metropolregion Rhein-Neckar · [a.eckerlin@gmx.de](mailto:a.eckerlin@gmx.de) · [LinkedIn](#) · [GitHub](#)

---

## Profil

Halluzinierte Literaturverzeichnisse kosten Journals und Verlage Stunden der Prüfung. Ich habe eine App gebaut, die diese Einträge per API gegen Fachdatenbanken abgleicht: Aus 2 Stunden Handarbeit werden zehn Minuten.

Als Verantwortlicher für digitale Barrierefreiheit habe ich unser laufendes Programm auf BFSG-Konformität umgestellt – vor der gesetzlichen Frist.

Als Senior Managing Editor habe ich für dieses Fachprogramm bisher rund 180 Titeln produziert. In dieser Zeit habe ich mit Markdown-Publishing-Pipelines experimentiert und mehrere KI-Werkzeuge für die Redaktion entwickelt – ein Word-Add-In zur Textkorrektur mit lokalem Sprachmodell, ein RAG-System für Dokumentenrecherche, die genannte Literaturprüfer-App.

Germanistik, korpuslinguistische Forschung mit der Twitter-API, Verlagslektorat, KI-gestützte Anwendungsentwicklung und redaktionelle Automatisierung, eigene KI-Tools. Was wie Umwege aussieht, ist meine Natur: Ich arbeite mich schnell in fremde Felder ein und verbinde sie, stelle Kreuzbezüge her. Wie könnte man X oder Y besser lösen?

---

## Ausgewählte Projekte

### Literaturprüfer

**Problem:** Halluzinierte oder falsch zitierte Quellen in Literaturverzeichnissen aufzuspüren kostet im Lektorat Stunden Handarbeit.

**Lösung:** macOS-App, die jeden Eintrag über einen Langdock-Agenten per API gegen Fachdatenbanken abgleicht und fehlerhafte oder erfundene Titel automatisch markiert (Code nicht öffentlich).

**Ergebnis:** Aus rund zwei Stunden Prüfung wurden etwa zehn Minuten. Im Team eingeführt und von drei Kolleg:innen genutzt.

### Aldit

**Problem:** Wiederkehrende Textkorrekturen binden Zeit im Lektorat — und Cloud-KI ist bei unveröffentlichten Manuskripten keine Option.

**Lösung:** Word-Add-in (Office.js), das Texte satzweise mit einem lokalen Sprachmodell über Ollama prüft und Verbesserungen als Kommentare einfügt, statt den Text automatisch zu ändern.

**Ergebnis:** Die Daten bleiben vollständig auf dem Rechner; die Autor:innen behalten die Kontrolle über jede einzelne Änderung.

**Stack:** JavaScript, Office.js, Webpack; lokale LLMs über Ollama (z. B. qwen2.5)

[LinkedIn-Post zu Aldit](#) · [Aldit auf GitHub](#)

### RAG-Bot for PDFs

**Problem:** Antworten in umfangreichen PDF-Sammlungen zu finden ist mühsam — und die Dokumente sollen das Haus nicht verlassen.

**Lösung:** Lokaler RAG-Bot, der Text aus PDFs extrahiert, in Embeddings überführt, mit FAISS durchsuchbar macht und Fragen über ein lokales LLM beantwortet.

**Ergebnis:** Beantwortet Fragen zu mehreren PDFs mit Belegstellen — vollständig offline, wahlweise per Weboberfläche oder Konsole.

**Stack:** Python, PyMuPDF, SentenceTransformers, FAISS, Ollama (Mistral/Nous-Hermes), Gradio  
[RAG-Bot for PDFs auf GitHub](#)

### **autodocx (Single-Source-Publishing, prototypisch)**

**Problem:** Print- und Digitalfassungen aus einer Quelle erzeugen, ohne Inhalte doppelt pflegen zu müssen.

**Lösung:** Prototyp einer Single-Source-Publishing-Pipeline auf Basis von XML/XSLT, die aus einer strukturierten Quelle mehrere Ausgabeformate erzeugt.

**Ergebnis:** Funktionsfähig aufgebaut, aber nicht produktiv ausgerollt — Grundlage für eine überarbeitete Pipeline, an der ich weiterarbeite.

**Stack:** XML, XSLT, Python

[autodocx auf GitHub](#)

## **Berufserfahrung**

---

### **Senior Managing Editor | Lektorat und Produktion**

Carl-Auer Systeme Verlag GmbH, Heidelberg · November 2021 – heute

- Produktion und Projektmanagement von bisher rund 180 Titeln aus Wissenschaft, Beratung und Organisationsentwicklung inkl. ePub-Produktion.
- Produktion auf barrierefreie digitale Ausgaben nach BFGG umgestellt.
- KI-Werkzeuge für die Redaktion selbst entwickelt und im Team eingeführt: App zur automatisierten Prüfung von Literaturverzeichnissen. Nutzung durch 3 Kolleg:innen. Zwei weitere Werkzeuge entwickelt: ein Word-Add-In zur Textkorrektur und einen lokalen RAG-Bot für PDF-Recherche (Ollama).
- Redaktionelle Workflows für Print und Digital neu erarbeitet.
- Crossfunktionale Steuerung zwischen Redaktion, Herstellung, Design, IT und externen Autor:innen.

### **Wissenschaftlicher Mitarbeiter**

Ruprecht-Karls-Universität Heidelberg · März 2021 – Oktober 2021

Im Forschungsprojekt "Culture Wars" der Universität Heidelberg habe ich über die Twitter-API Sprachdaten extrahiert und korpuslinguistisch untersucht.

### **Werkstudent**

Carl-Auer Systeme Verlag GmbH, Heidelberg · Oktober 2019 – September 2021

## **Ausbildung**

---

### **Master of Arts (MA), Germanistik**

Ruprecht-Karls-Universität Heidelberg · 2019 – 2021

### **Bachelor of Arts (BA), Germanistik und Geschichte**

Ruprecht-Karls-Universität Heidelberg · 2015 – 2019

## **Fortbildungen und Seminare**

---

- AI Literacy: KI-Kompetenz für Medien
- XML-Grundlagen
- UX Strategy Fundamentals

- Systemische Organisationsberatung (Grundkurs, simon weber friends)
- NLP-Practitioner

## Kenntnisse & Kompetenzen

---

- KI-Werkzeuge bauen & einführen: lokale LLMs (Ollama), RAG, Prompt Engineering, KI Prompting
- Prozessautomatisierung redaktioneller und dokumentenbasierter Abläufe
- Office-/Word-Add-ins (Office.js), Python-Tooling
- Digitale Barrierefreiheit (BFSG, WCAG)
- Wissensmanagement & Wissensvermittlung
- Team- & Stakeholder-Koordination, crossfunktionale Zusammenarbeit
- Verlagsmanagement & redaktionelle Koordination
- Digitale Medien & Content-Strategie
- Projektmanagement
- Erprobt / prototypisch: XML, XSLT, Single-Source-Publishing (aufgebaut, nicht produktiv ausgerollt)

## Sprachen

---

- Deutsch — Muttersprache
- Englisch — Verhandlungssicher (C1)
- Spanisch — Gute Kenntnisse